API Data Ingestion Pipeline

Setup and Run Guide: install, configure, run, and check that it works.

What you need

- Python 3.10 or newer (3.11 or 3.12 is recommended). Check with `python --version` (use `python3` if `python` is not found).
- Git, to download the project.
- Two terminal windows.
- Windows, macOS or Linux. No cloud account is needed, and after the install no internet connection is needed.

Step 1: Get the project

```
git clone <repository-url>
cd <project-folder>
```

Replace `<repository-url>` with the address of the project's GitHub repository. Run every later command from the folder that contains `pyproject.toml`. If you see another folder with the same name inside it, go into that one until you can see `pyproject.toml`.

Step 2: Install the dependencies

Windows (PowerShell)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
```

macOS / Linux

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
```

When the environment is active, your prompt starts with `(.venv)`. If PowerShell says that running scripts is disabled, run this once and then activate again:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Step 3: Configure the project

The project works with its default settings, so you can skip this step for a first run. The defaults are in `config/config.yaml`:

Setting	Default	What it does
`base_url`	`http://127.0.0.1:8000`	Address of the mock API
`timeout_seconds`	`10`	How long to wait for one response
`max_retries`	`4`	How many times a failed request is retried
`backoff_seconds`	`0.2`	Starting wait between retries (it doubles each time)
`requests_per_second`	`20`	Request pacing, to avoid overloading the source
`page_size`	`250`	Records per page

To change the address or token without editing the file, set environment variables in your terminal before you start. The names are `API_BASE_URL`, `API_TOKEN`, `API_TIMEOUT_SECONDS`, `API_MAX_RETRIES`, `API_BACKOFF_SECONDS` and `API_REQUESTS_PER_SECOND`.

The token has two sides. The client sends `API_TOKEN`, and the mock API server checks it against `MOCK_API_TOKEN`. Both default to `local-demo-token`. If you change one, change the other to the same value, in both terminals. For example:

```
$env:API_TOKEN = "my-token"
$env:MOCK_API_TOKEN = "my-token"
```

On macOS or Linux, use `export API_TOKEN=my-token` and `export MOCK_API_TOKEN=my-token`. This is a demo token, not a real secret. Never put real credentials in this project.

Step 4: Run the project

Terminal 1: start the mock API and leave it running.

```
python -m pipeline.cli serve --host 127.0.0.1 --port 8000
```

Terminal 2: open it in the same folder, activate the virtual environment again, then run:

```
python -m pipeline.cli generate --count 100 --seed 42 --output data/sample/orders.json
python -m pipeline.cli ingest
python -m pipeline.cli ingest --start-page 1
python -m pipeline.cli reconcile
python -m pipeline.cli benchmark-cmd
python -m pytest -rA
```

- `generate` creates 100 test records. It adds some bad records on purpose, to show validation.
- `ingest` reads every page from the mock API and loads the data.
- `ingest --start-page 1` runs the same input again, to show that a rerun creates no duplicates.
- `reconcile` checks that the numbers add up.
- `benchmark-cmd` compares SQLite and Parquet timing.
- `python -m pytest -rA` runs the 21 automated tests.

Restart the mock API in Terminal 1 every time you generate new data.

Step 5: Check that it worked

You should see these numbers in the log lines and reports:

	Fetched	Accepted	Rejected	Skipped (already loaded)
First `ingest`	100	93	6	1
Replay with `--start-page 1`	100	0	6	94

After both runs, `reconcile` should show:

- `target_rows: 93`
- `skipped_existing_across_runs: 95`
- `unexplained_records: 0`
- `source_reconciles: true` and `target_matches_accepted: true`
- `duplicate_target_keys: 0`

The 6 rejected records are the bad records that the generator adds on purpose. Each one is saved in quarantine with its reason. The tests should end with `21 passed`. These numbers are for a fresh start with the 100-record command above.

Where the results are

File	What is inside
`data/output/pipeline.db`	Target data, checkpoints, run history, page ledger and quarantine records
`data/output/raw/<run_id>/page_*.json`	The original API responses
`data/output/parquet/orders.parquet`	The analytical snapshot
`reports/run_<run_id>.json`	Metrics for each run
`reports/reconciliation.json`	Reconciliation results
`reports/benchmark.json`	SQLite and Parquet timing

Start over

Do this when you want to run the data again from the beginning. Stop the mock API first, then run:

Windows (PowerShell)

```
Remove-Item data\output\pipeline.db* -Force
Remove-Item data\output\raw -Recurse -Force
Remove-Item reports\run_*.json
```

macOS / Linux

```
rm -f data/output/pipeline.db*
rm -rf data/output/raw
rm -f reports/run_*.json
```

Then start the mock API again.

Optional: try a failure and a restart

1. Start over (see above), and start the mock API.
2. In `config/config.yaml`, change `page_size: 250` to `page_size: 25`, so the 100 records become 4 pages.
3. In Terminal 2 run the commands below. On Windows, use `curl.exe` instead of `curl`.

```
curl -X POST http://127.0.0.1:8000/__control/reset
curl -X POST http://127.0.0.1:8000/__control/fail-always/2
python -m pipeline.cli ingest
```

Page 1 is saved. Page 2 fails after several retries, and the run ends with a 503 error. The checkpoint stays at page 2. Now remove the fault and run again:

```
curl -X POST http://127.0.0.1:8000/__control/reset
python -m pipeline.cli ingest
```

The second run starts at page 2 and finishes pages 2 to 4. It does not fetch page 1 again. `reconcile` should then show `target_rows: 93`. Change `page_size` back to `250` when you are done.

Optional: larger run

Start over, then generate 100,000 records, restart the mock API, and run:

```
python -m pipeline.cli generate --count 100000 --seed 42 --output data/sample/orders.json --no-edge-cases
python -m pipeline.cli ingest
python -m pipeline.cli reconcile
```

`reconcile` should show `target_rows: 100000` and `unexplained_records: 0`. On the test machine this took about 30 seconds. Let it finish without stopping it. Afterwards, generate the 100-record sample again.

Troubleshooting

Problem	What to do
PowerShell says running scripts is disabled	Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then activate the environment again.
`No module named pipeline`	The virtual environment is not active, or the install did not finish. Activate it, then run `python -m pip install -e ".[dev]"` again.
A config file or path is not found	You are not in the folder that contains `pyproject.toml`. Go to that folder.
`Connection refused` during `ingest`	The mock API is not running. Start it in Terminal 1 (Step 4), and check that the address in `config/config.yaml` matches.
`401 Unauthorized`	The tokens do not match. `API_TOKEN` and `MOCK_API_TOKEN` must have the same value.
Port 8000 is already in use	Start the server with `--port 8001`, and set `API_BASE_URL` to `http://127.0.0.1:8001` in Terminal 2.
`ingest` shows `fetched=0`	The checkpoint is already past the last page. Run `python -m pipeline.cli ingest --start-page 1`, or start over.
Numbers differ from Step 5	The data or outputs come from an earlier run. Start over, generate the 100 records again, restart the mock API, and run again.
pytest crashes at startup with an error about `_str_cache`	Install an older pytest: `python -m pip install "pytest>=8,<9"`.